

Project Synopsis & Design Document

For

CareerOS

CareerOS: An AI-Powered Career Intelligence Platform

Name: Vaibhav Sahu

Roll No.: 25225026

Table of Contents

1. Introduction	3
2. Project Objectives & SDG Alignment	3
2.1 Objectives	3
2.2 Alignment with UN Sustainable Development Goals (SDGs)	3
3. Existing System and Its Limitations	3
3.1 Key Limitations	4
4. Proposed System	4
4.1 Core Capabilities	4
4.2 What Makes It Different	4
5. Functional and Non-Functional Requirements	5
5.1 Functional Requirement Categories	5
5.2 Non-Functional Requirement Categories	5
6. System Design	5
6.1 UI Design	6
6.2 Database Design	8
6.3 Module / Component Architecture	10
6.4 Algorithms / Flowcharts	11
7. Expected Outcomes	13
8. Specialization Concepts Used	14
9. GitHub Repository	14

1. Introduction

This document combines the Project Synopsis and the Design Document for CareerOS: An AI-Powered Career Intelligence Platform into a single reference: it states the project's objectives and SDG alignment, analyzes the existing system's limitations, describes the proposed system and its requirements, and then details the system design — UI, database, modules, and algorithms — that will be used to build it.

2. Project Objectives & SDG Alignment

2.1 Objectives

- To design and develop a unified platform that consolidates resume analysis, skill-gap identification, learning-path guidance, mock interviews, and portfolio evaluation into a single AI-driven system.
- To build an AI layer that retains a persistent “Career Memory” of each student, enabling recommendations that improve in relevance over repeated use rather than resetting each session.
- To provide role-based dashboards for Students, Recruiters, Placement Officers, Faculty, and Administrators so each stakeholder gets analytics relevant to their responsibilities.
- To generate an explainable Career Readiness Score that quantifies employability using resume quality, skill-gap closure, interview performance, and portfolio signals.
- To demonstrate applied use of Generative AI, NLP, retrieval-augmented generation, and full-stack cloud deployment within a one-semester academic timeline.

2.2 Alignment with UN Sustainable Development Goals (SDGs)

CareerOS is designed to directly support two United Nations Sustainable Development Goals:

SDG	How CareerOS Contributes
SDG 4 — Quality Education	CareerOS personalizes learning roadmaps to each student's actual skill gaps, improving the relevance and accessibility of the learning resources a student is guided toward, rather than leaving them to discover a path unassisted.
SDG 8 — Decent Work and Economic Growth	By improving resume quality, interview readiness, and skill-role fit, CareerOS aims to improve students' employability outcomes and helps recruiters and placement cells match candidates to roles more efficiently.

3. Existing System and Its Limitations

Students currently piece together their career preparation from multiple disconnected tools: resume builders (e.g., Canva, Novoresume), generic ATS checkers, standalone interview-prep apps, MOOC platforms for learning paths, and manual tracking of certificates and GitHub activity. Placement cells and

recruiters, in turn, rely on manually collected spreadsheets or generic Applicant Tracking Systems that were not designed for an academic institution's placement workflow.

3.1 Key Limitations

- Fragmentation: no single tool spans resume, skills, interviews, and portfolio together, so a student's overall readiness is never visible in one place.
- No memory across sessions: each tool treats every interaction as a fresh start; a generic AI chatbot used for career advice does not remember prior context, so guidance does not compound or improve over time.
- Generic, not personalized: most resume/ATS tools apply one-size-fits-all scoring rules that do not account for a student's specific target role or skill trajectory.
- Manual placement-cell reporting: department-level readiness tracking is typically done in spreadsheets, which do not scale and go stale quickly.
- No explainability: existing AI resume tools frequently return a score without a clear, auditable breakdown of what drove it, which limits students' ability to act on the feedback.

4. Proposed System

CareerOS proposes a single, AI-driven “Career Operating System” that unifies the fragmented tools above into one platform, and adds a persistent Career Memory layer so that the system's guidance to a student improves as it learns more about them over time.

4.1 Core Capabilities

- Resume upload, AI-based parsing, ATS scoring, and structured, section-by-section improvement feedback.
- Skill-gap analysis against a chosen target role, with a personalized, sequenced learning roadmap.
- An AI Career Mentor that uses retrieval-augmented generation (RAG) over each student's own Career Memory and documents to give grounded, personalized guidance.
- AI-generated mock interviews with structured feedback and performance tracking across sessions.
- GitHub portfolio analysis and project recommendations aligned with a student's skill gaps.
- A composite, explainable Career Readiness Score with a visible breakdown of contributing factors.
- Role-based dashboards for Recruiters (job posting, candidate search, AI ranking), Placement Officers (cohort readiness analytics), Faculty (assigned-student monitoring), and Admins (user and AI-service management).

4.2 What Makes It Different

Unlike a thin wrapper around a general-purpose chatbot, CareerOS's differentiators are purpose-built components: a Skill Gap Prediction Engine, a Career Readiness Score aggregator, a GitHub Portfolio Analyzer, and a Career Memory Engine backed by a vector store — each independently explainable and testable, and each addressing a specific limitation of the fragmented existing tools described in Section 3.

5. Functional and Non-Functional Requirements

A complete, numbered requirements catalogue (with per-requirement priority tags) is provided in the companion Software Requirements Specification (SRS) document. The summary below highlights the requirement categories.

5.1 Functional Requirement Categories

Module	Summary
Authentication & User Management	Registration, login, RBAC across 5 roles
Career Profile & Resume	Profile management, resume upload/parsing, ATS scoring
Skill Gap & Learning Roadmap	Target-role comparison, personalized roadmap generation
AI Career Mentor & Career Memory	Persistent, RAG-grounded conversational guidance
Mock Interview	AI-generated questions and structured feedback
GitHub Portfolio & Readiness Score	Portfolio analysis, certificate tracking, composite score
Recruiter Module	Job posting, candidate search, AI-assisted ranking
Placement Cell & Faculty Modules	Cohort/departments analytics, assigned-student monitoring
Admin Module	User management, AI-service monitoring, system logs

5.2 Non-Functional Requirement Categories

- Performance: sub-500ms for standard API calls; AI-backed responses begin returning within 5 seconds.
- Security: HTTPS everywhere, hashed credentials, RBAC enforced server-side, minimized PII sent to third-party LLM APIs.
- Reliability: graceful degradation of student-facing features if the AI service is temporarily unavailable.
- Scalability: independently deployable, containerized backend and AI microservice.
- Usability: first-time users can complete profile creation and a first resume analysis within a 10-minute session.
- Maintainability & Portability: modular, layered architecture; deployable to AWS, Azure, or GCP via Docker without code changes.

6. System Design

This section covers the UI design, database design, module architecture, and core algorithms that implement the proposed system described in Section 4.

6.1 UI Design

CareerOS uses a role-based dashboard pattern: every user lands on a dashboard scoped to their role, reached through a shared top navigation and a role-specific sidebar. The layout below illustrates the Student Dashboard, the most feature-dense screen; other role dashboards (Recruiter, Placement Officer, Faculty, Admin) follow the same shell with role-appropriate sidebar items and cards.

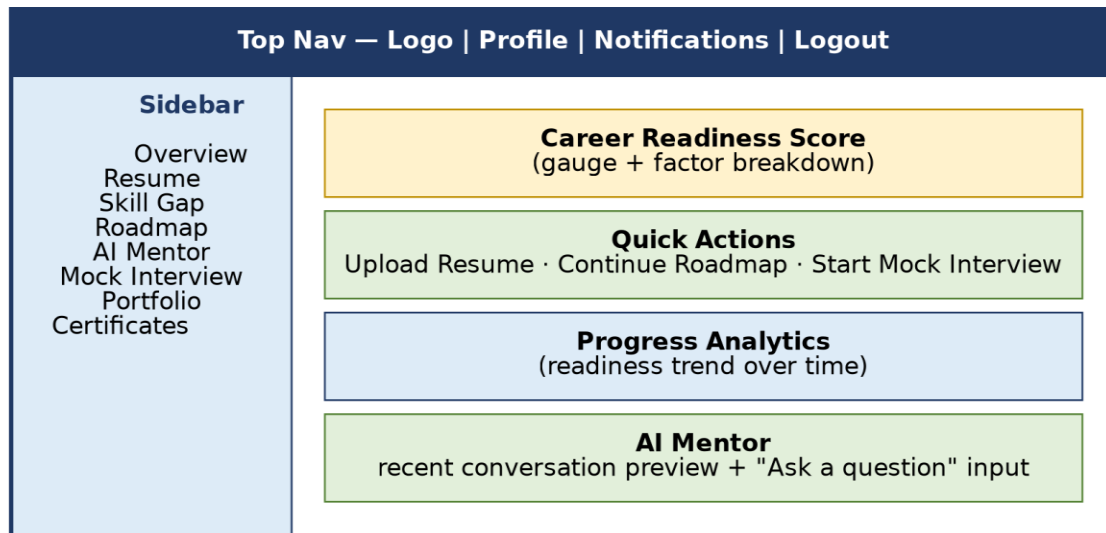


Figure 1: Student Dashboard — wireframe layout

Key Screens

Screen	Purpose / Key Elements
Login / Register	Email + password (or SSO) form; role is assigned by an Admin, not self-selected at signup.
Student Dashboard	Readiness Score gauge, quick-action cards, progress analytics, AI Mentor preview (Figure 1).
Resume Upload & Analysis	Drag-and-drop upload, parsed-field preview, ATS score with breakdown, section-by-section feedback.
Skill Gap & Roadmap	Target-role selector, missing-skill list, sequenced roadmap with progress checkboxes.
AI Mentor Chat	Conversational chat panel with message history and a persistent context indicator.
Mock Interview	Role/difficulty selector, question panel, answer input, post-session structured feedback.
Portfolio & Certificates	GitHub connection status, repository analysis summary, certificate upload/list.
Recruiter — Candidate Search	Filter panel (skills, readiness score) + ranked candidate list with AI rationale.
Placement Officer — Analytics	Cohort readiness distribution, department comparison charts, export controls.

Screen	Purpose / Key Elements
Admin Console	User management table, AI-service health panel, system log viewer.

UI Design Principles

- Consistency: shared top-nav and card/table components across all five role dashboards.
- Progressive disclosure: summary cards link out to detail screens rather than showing everything at once.
- Explainability: any AI-derived number (ATS score, Readiness Score, ranking) is always shown with a visible factor breakdown, never as a bare number.
- Responsiveness: layouts collapse the sidebar into a top drawer on tablet-width viewports.

6.2 Database Design

PostgreSQL is the relational system of record. The core entities and relationships are shown below; JSON columns are used for AI-derived structured content (parsed resume fields, skill lists, feedback objects) to keep the schema flexible while the AI output format evolves during development.

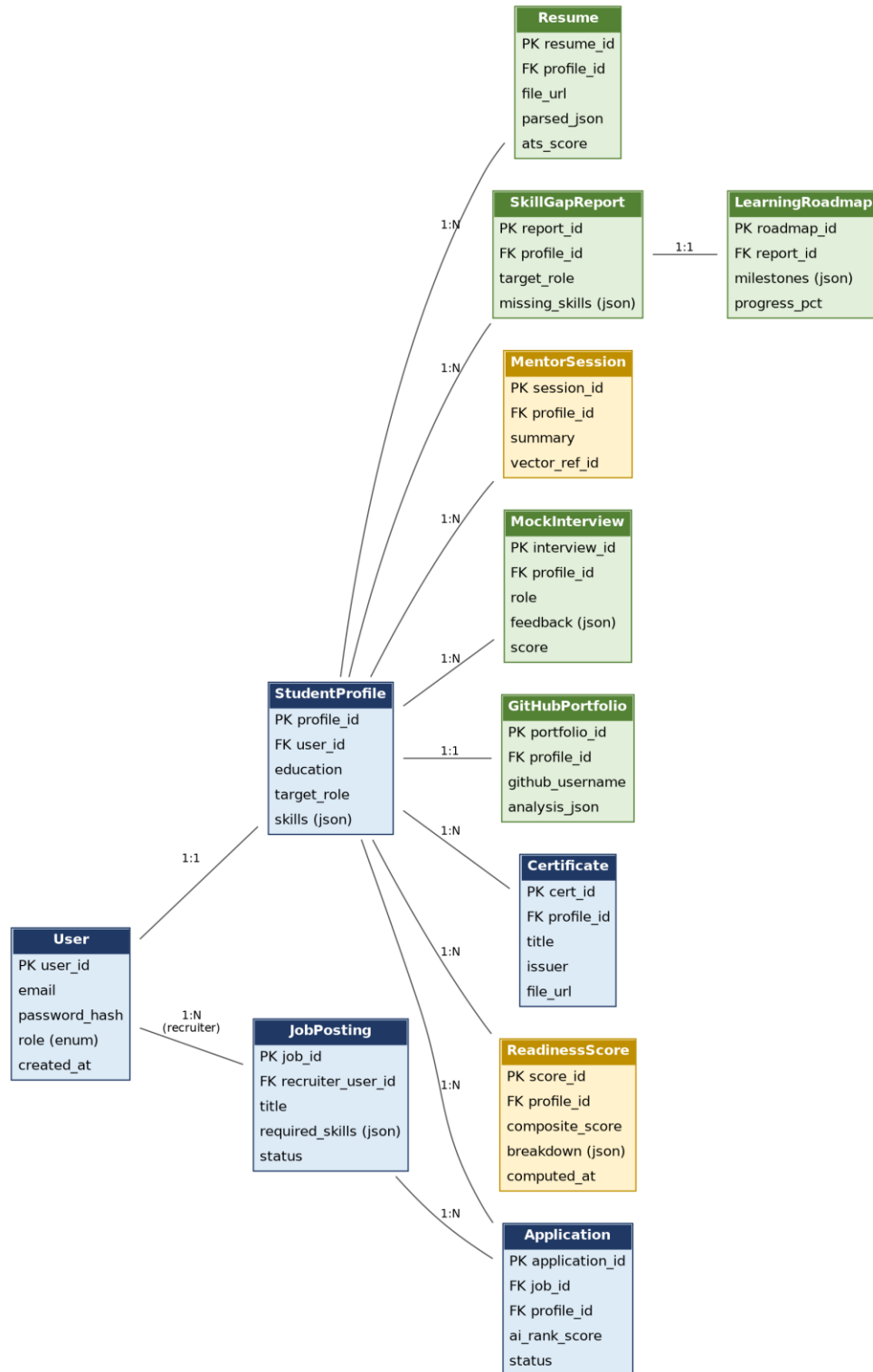


Figure 2: Entity-Relationship diagram (core schema)

Notes on the Schema

- User is the single authentication entity for all five roles; role is stored as an enum and drives RBAC checks in the API layer.
- StudentProfile is 1:1 with a Student-role User and is the anchor that SkillGapReport, MentorSession, MockInterview, GitHubPortfolio, Certificate, ReadinessScore, and Application all reference.
- MentorSession stores only a summary and a reference ID into the vector store (vector_ref_id) — the full embeddings live in the vector database, not in PostgreSQL, keeping the relational schema lean.
- JobPosting and Application support the Recruiter flow; Application.ai_rank_score is written by the AI ranking service, not computed client-side.
- Redis (not shown in the ER diagram) is used for session storage, response caching, and background job queues (e.g., resume-parsing jobs), and holds no data considered a system of record.

6.3 Module / Component Architecture

The system is layered into a client tier, an application tier, an AI services tier, and a data tier, with the Node.js backend acting as the single entry point that the frontend talks to — the frontend never calls the AI microservice directly.

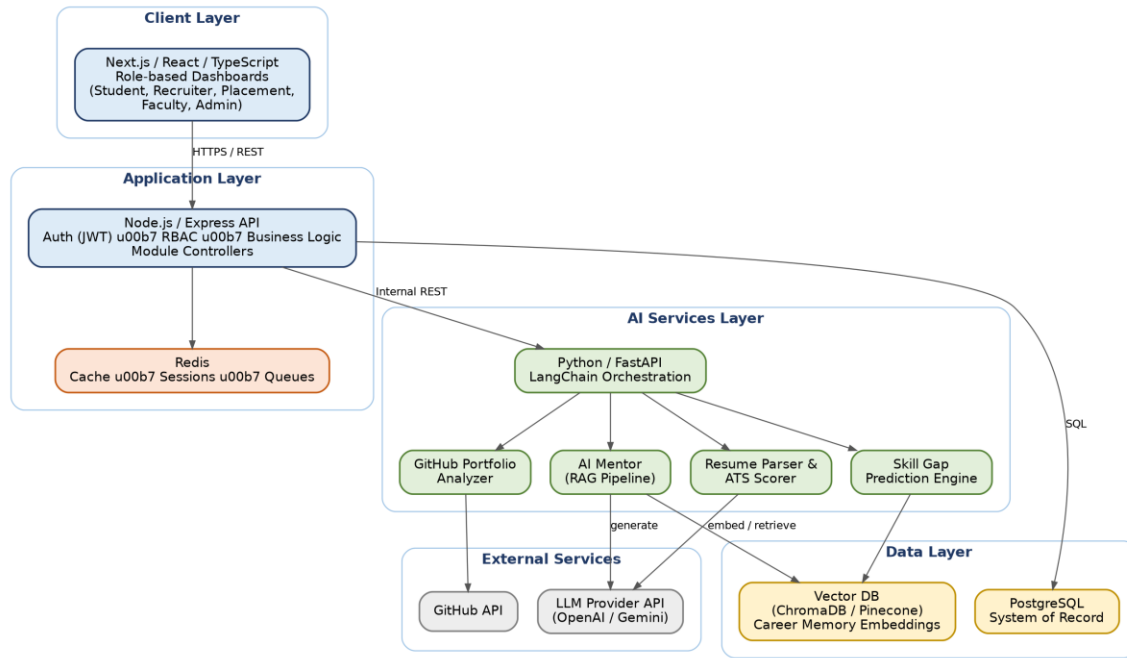


Figure 3: System architecture / module diagram

Layer Responsibilities

Layer	Responsibility
Client Layer	Next.js/React/TypeScript app rendering the five role-based dashboards; handles client-side state and calls the backend over HTTPS/REST.
Application Layer	Node.js/Express API: authentication (JWT), RBAC enforcement, module controllers (Student, Recruiter, Placement, Faculty, Admin), and Redis-backed caching/sessions/queues.
AI Services Layer	Python/FastAPI microservice orchestrated with LangChain, exposing internal endpoints for resume parsing, skill-gap prediction, the AI Mentor's RAG pipeline, and GitHub portfolio analysis.
Data Layer	PostgreSQL for relational data; a vector database (ChromaDB or Pinecone) for Career Memory embeddings and RAG retrieval.
External Services	Third-party LLM provider API (OpenAI or Gemini) and the GitHub API.

6.4 Algorithms / Flowcharts

Skill Gap Analysis

Compares a student's parsed resume skills against the required-skill profile for a chosen target role, and generates a roadmap for any gap found.

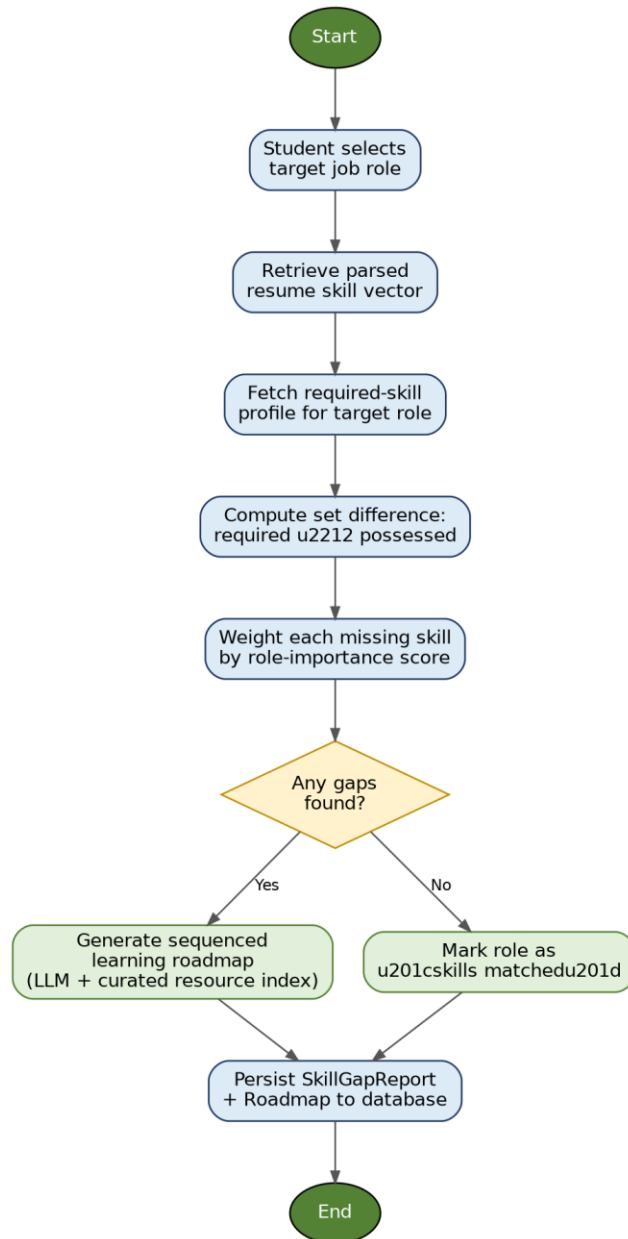


Figure 4: Skill Gap Analysis algorithm

Career Readiness Score

A weighted composite of four signals, recomputed whenever a student's profile-affecting data changes. Weights are configurable and documented in the breakdown returned to the client, so the score is auditable rather than a black box.

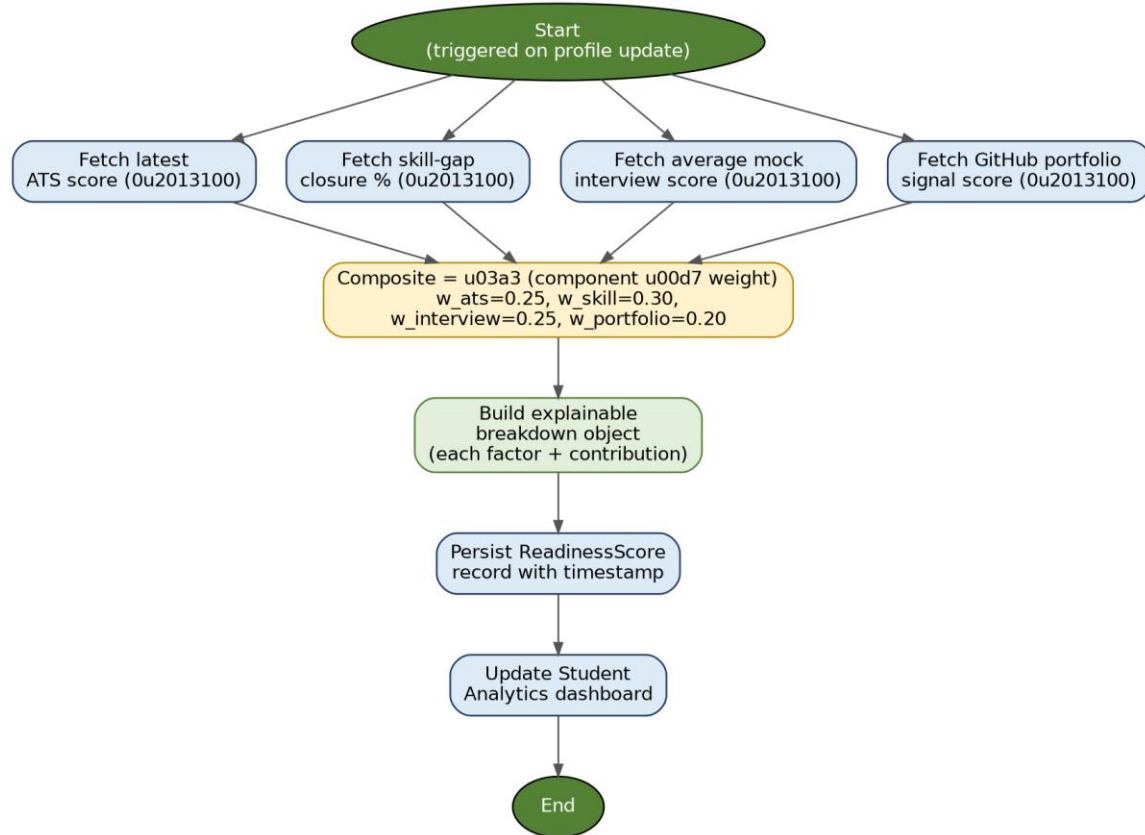


Figure 5: Career Readiness Score computation

AI Mentor — Retrieval-Augmented Generation (RAG)

Grounds every AI Mentor response in the student's own Career Memory rather than relying on the LLM's un-contextualized output, and writes a summary of each exchange back into that memory so future responses stay personalized.

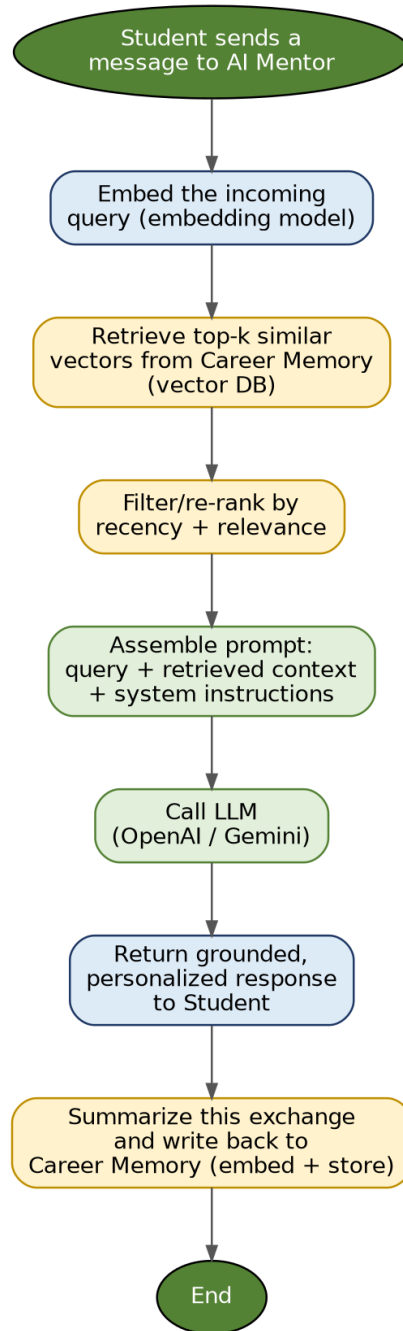


Figure 6: AI Mentor RAG retrieval pipeline

7. Expected Outcomes

- A working MVP demonstrating end-to-end student flow: profile creation → resume analysis → skill gap → roadmap → mentor guidance → mock interview → readiness score.
- A working Recruiter flow: job posting → candidate search → AI-assisted ranking.
- A working Placement Officer view: cohort-level readiness and skill-gap analytics.
- A deployed, containerized system accessible via a public URL for demonstration and viva.

- Complete academic documentation: Synopsis, SRS, Design Document, Presentation, testing report, and user manual.
- A GitHub repository with weekly commit history evidencing iterative development.

8. Specialization Concepts Used

- Generative AI & NLP: LLM-based resume parsing, interview question generation, and conversational mentoring.
- Retrieval-Augmented Generation (RAG): grounding AI Mentor responses in a student's own Career Memory and documents via a vector database.
- Full Stack Development: Next.js/React/TypeScript frontend, Node.js/Express backend, PostgreSQL/Redis data layer.
- Cloud Computing & DevOps: Dockerized microservices deployed on a cloud provider (AWS/Azure/GCP).
- Recommendation Systems (as applicable): skill-gap-driven learning-path and project recommendations.

9. GitHub Repository

Repository link: <https://github.com/uhoh-vaibhav/CareerOS>

The repository will maintain a weekly commit history throughout the semester as required by the project evaluation criteria, along with a README documenting setup instructions and architecture.